

Comparaciones y Decisiones en JavaScript

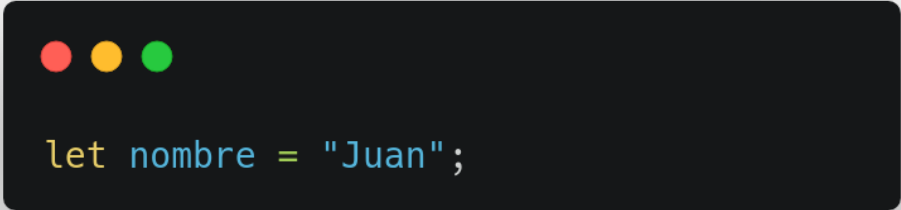
Guardando Valores en Variables y Operadores Básicos

Cuando empiezas a programar, una de las primeras cosas que necesitas es **guardar información** para poder utilizarla después en tu código. Para eso usamos algo llamado **variables**. Las variables son como cajas donde guardas cosas que luego puedes usar, cambiar, o consultar más tarde. En JavaScript, hay tres maneras principales de declarar estas cajas: usando `var`, `let` y `const`. Cada una tiene sus particularidades, veámos cómo y cuándo usarlas.

¿Qué son las Variables?

Las variables nos permiten almacenar datos, como números, palabras o resultados de cálculos. Imagina que estás haciendo una receta y quieres guardar cuántos ingredientes necesitas. En lugar de tener que recordar ese número en todo momento, lo anotas en una variable.

Por ejemplo, si quieres guardar el nombre de un usuario en tu programa, puedes hacerlo así:



```
let nombre = "Juan";
```

image.png

Aquí, hemos creado una variable llamada `nombre` y guardado el valor `"Juan"` en ella. Cada vez que necesites usar ese nombre en tu programa, simplemente llamas a la variable `nombre`. Las variables son clave para hacer que tu código sea dinámico y flexible, porque los valores que guardan pueden cambiar en cualquier momento.

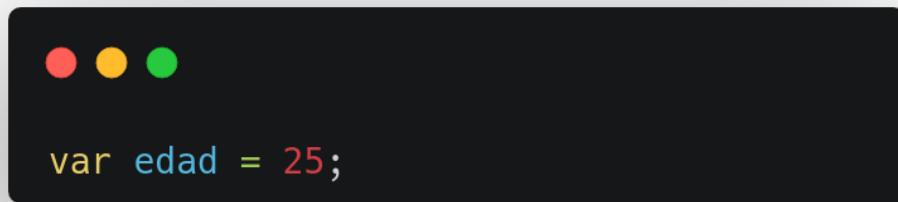
`var`, `let`, y `const`: ¿Qué son y cuándo usarlas?

En JavaScript, hay tres formas de declarar una variable: var, let y const. A simple vista, parecen hacer lo mismo: guardar valores, pero tienen diferencias importantes.

1. var

Esta es la forma más antigua de declarar una variable en JavaScript. Aunque sigue funcionando, hoy en día es menos recomendable usar var en la mayoría de los casos porque tiene algunas características que pueden hacer que el código se comporte de manera inesperada.

Ejemplo:



```
var edad = 25;
```

image.png

¿Por qué no recomendamos usar var? En muchas situaciones, var puede provocar que las variables se comporten de formas que no son fáciles de entender. Más adelante en tu aprendizaje, veremos por qué. Por ahora, simplemente es mejor que uses let o const, que son más modernas y seguras.

2. let

Esta es la forma más común y recomendada para declarar variables que van a cambiar su valor. Si sabes que el valor de la variable puede modificarse más adelante, usa let.

Ejemplo:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains two lines of JavaScript code: `let contador = 0;` and `contador = contador + 1; // Ahora contador vale 1`.

```
let contador = 0;

contador = contador + 1; // Ahora contador vale 1
```

image.png

Con `let`, puedes estar seguro de que la variable solo funcionará donde la declares, lo cual ayuda a evitar errores en tu código.

3. `const`

Si sabes que una variable nunca va a cambiar de valor, es mejor usar `const`. Como su nombre lo sugiere, `const` significa constante, lo que implica que una vez que asignes un valor, ese valor no se puede modificar.

Ejemplo:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains one line of JavaScript code: `const pi = 3.1416;`.

```
const pi = 3.1416;
```

image.png

En este caso, como `pi` no va a cambiar nunca, usamos `const`. Esto también es útil porque te protege de cometer errores accidentalmente al intentar modificar una variable que no deberías.

¿Cuál es la ventaja de usar `let` y `const`?

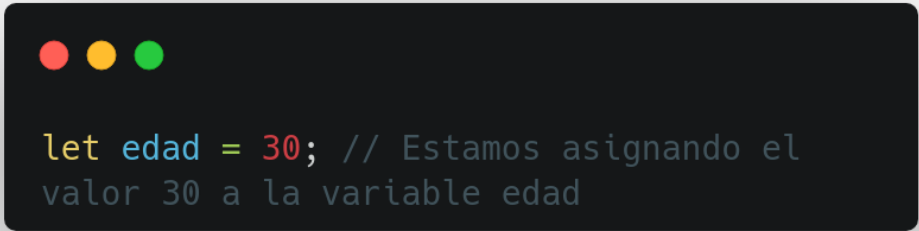
La principal ventaja de usar `let` y `const` en lugar de `var` es que son más predecibles y fáciles de controlar. Te permiten tener más control sobre dónde y cómo se pueden usar las variables, lo cual te ayuda a escribir código más seguro y evitar errores que podrían ser difíciles de encontrar. En general, usa `let` para variables que cambiarán y `const` para las que no.

Operadores Básicos

Una vez que tienes variables para guardar datos, necesitas poder hacer cosas con ellas, como sumarlas, restarlas o compararlas. Para esto, JavaScript tiene **operadores** que te permiten manipular esos datos.

1. Operadores de Asignación. El operador de asignación más común es el signo igual `=`, que se usa **para asignar un valor a una variable**.

Ejemplo:



```
let edad = 30; // Estamos asignando el
valor 30 a la variable edad
```

image.png

2. Operadores Matemáticos. Además de asignar valores, puedes realizar operaciones matemáticas con las variables. JavaScript tiene operadores para sumar, restar, multiplicar y dividir.

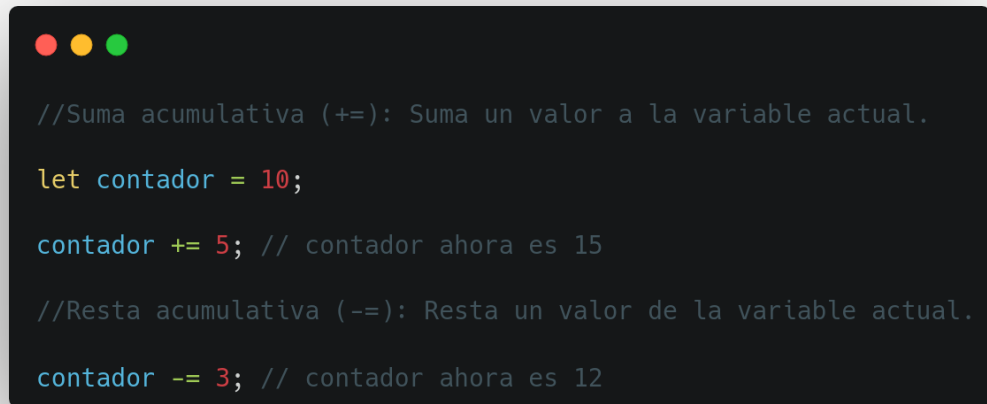


```
Suma (+): Suma dos valores.  
  
let a = 5;  
let b = 10;  
  
let suma = a + b; // suma es 15  
  
//Resta (-): Resta un valor de otro.  
  
let resta = b - a; // resta es 5  
  
//Multiplicación (*): Multiplica dos valores.  
  
let producto = a * b; // producto es 50  
  
//División (/): Divide un valor por otro.  
  
let division = b / a; // division es 2  
  
//Módulo (%): Devuelve el residuo de una división.  
  
let residuo = b % a;  
// residuo es 0, porque 10 dividido entre 5 no deja residuo
```

image.png

Los operadores matemáticos son útiles para realizar cálculos en tus programas. Por ejemplo, podrías usarlos para hacer una calculadora básica, o para sumar precios de productos en una tienda en línea.

3. Operadores Combinados. JavaScript también tiene operadores que combinan la asignación con una operación matemática. Estos operadores te permiten hacer cálculos más rápido.



```
//Suma acumulativa (+=): Suma un valor a la variable actual.  
let contador = 10;  
contador += 5; // contador ahora es 15  
  
//Resta acumulativa (-=): Resta un valor de la variable actual.  
contador -= 3; // contador ahora es 12
```

image.png

Estos operadores son muy útiles cuando necesitas modificar el valor de una variable repetidamente, como en un ciclo o al contar elementos.

Las **variables** son el pilar básico de cualquier programa, y en JavaScript puedes declararlas de diferentes maneras (var, let, const), dependiendo de si el valor puede cambiar o no. Además, los **operadores** te permiten manipular los datos almacenados en esas variables, ya sea para hacer cálculos o combinar resultados.

A medida que avances en tu aprendizaje, entender cómo y cuándo usar variables será fundamental para escribir programas eficientes y claros. Y recuerda, siempre que puedas, usa let o const para declarar tus variables, ya que te ayudarán a evitar problemas más adelante.

Operadores de Comparación y Lógicos

En programación, necesitamos comparar valores constantemente para tomar decisiones. JavaScript nos ofrece herramientas llamadas **operadores de comparación y operadores lógicos** para evaluar condiciones y combinar varias de ellas. En esta lectura, vamos a centrarnos en cómo funcionan estos operadores y cuándo usarlos.

Operadores de Comparación

Los operadores de comparación nos permiten comparar dos valores y devolver un resultado **true** (verdadero) o **false** (falso), según el resultado de la comparación. Veamos los más comunes.

1. Igualdad Débil (==)

- El operador `==` compara dos valores, pero **convierte los tipos de dato** si es necesario antes de hacer la comparación. Esto significa que dos valores que parecen diferentes a simple vista pueden considerarse iguales porque JavaScript los transforma.

Ejemplo:

```
let a = 5;
```

```
let b = "5";
```

```
console.log(a == b); // true, porque JavaScript convierte el string "5" a número
```

En este caso, aunque `a` es un número y `b` es un string, JavaScript convierte `b` a número antes de compararlos, lo que devuelve `true`. Esta conversión automática puede ser confusa y producir resultados inesperados. Es por eso que el uso de `==` no es lo mejor, ya que es poco usado y no tan recomendable como `===`.

2. Igualdad Estricta (===)

- El operador `===` **compara los valores y también los tipos** de dato. Esto significa que solo devuelve `true` si los dos valores son exactamente iguales y del mismo tipo. Este es el operador que se recomienda usar en la mayoría de los casos porque evita las conversiones automáticas de tipos.

Ejemplo:

```
console.log(a === b); // false, porque uno es número y el otro es string
```

Aquí, como `a` es un número y `b` es un string, `===` devuelve `false` porque no hace conversiones automáticas. Es más seguro y predecible que `==`. Más adelante en esta lectura, haremos una aclaración de cuál es mejor usar en distintas situaciones.

3. Diferencia Débil (!=)

- Este operador compara dos valores y devuelve true si son diferentes, **ignorando el tipo de dato**.

Ejemplo:

```
let x = 5;  
let y = "5";  
  
console.log(x != y); // false, porque convierte "5" a número y los considera iguales
```

4. Diferencia Estricta (!==)

- Compara los valores y tipos de datos. Devuelve true si son diferentes en valor o tipo.

Ejemplo:

```
console.log(x !== y); // true, porque uno es número y el otro string
```

5. Mayor que (>) y Menor que (<)

- Compara si un valor es mayor o menor que otro.

Ejemplo:

```
let c = 10;  
let d = 8;  
  
console.log(c > d); // true  
console.log(c < d); // false
```

6. Mayor o igual que (>=) y Menor o igual que (<=)

- Compara si un valor es mayor o igual, o menor o igual que otro.

Ejemplo:

```
let z = 15;  
  
console.log(z >= 10); // true
```

¿Por Qué Es Mejor Usar ===?

Aunque JavaScript permite usar == para comparar valores con tipos diferentes, esto puede llevar a errores inesperados. Con ==, JavaScript trata de hacer una **conversión automática de tipos**, lo que puede producir resultados que no esperamos.

Ejemplo con ==:

```
console.log(0 == false); // true, porque JavaScript convierte `false` en `0`
```

```
console.log(0 === false); // false, porque no son del mismo tipo (número y booleano)
```

El uso de **igualdad estricta** (===) es más recomendable porque te asegura que la comparación tenga en cuenta **tanto el valor como el tipo de dato**. Esto te ayuda a evitar conversiones automáticas que podrían causar confusión o errores en tu programa.

Operadores Lógicos

Los **operadores lógicos** nos permiten combinar varias condiciones. Esto es útil cuando necesitamos que se cumplan múltiples reglas a la vez o cuando cualquiera de varias condiciones sea verdadera.

1. AND (&&)

- Devuelve true solo si ambas condiciones son verdaderas. Si alguna es falsa, devuelve false.

Ejemplo:

```
let edad = 20;
```

```
let tieneLicencia = true;
```

```
console.log(edad >= 18 && tieneLicencia); // true, porque ambas condiciones son verdaderas
```

2. OR (||)

- Devuelve true si al menos una de las condiciones es verdadera. Solo devuelve false si todas son falsas.

Ejemplo:

```
let esMayorDeEdad = false;
```

```
let esEstudiante = true;
```

```
console.log(esMayorDeEdad || esEstudiante); // true, porque una condición es verdadera
```

3. NOT (!)

- Este operador invierte el valor de una condición. Si es true, la convierte en false, y viceversa.

Ejemplo:

```
let esFinDeSemana = true;  
  
console.log(!esFinDeSemana); // false, porque invierte el valor
```

Combinando Operadores

Puedes combinar operadores de comparación y lógicos para hacer evaluaciones más complejas. Esto es muy útil cuando necesitas que se cumplan varias condiciones a la vez.

Ejemplo:

```
let edad = 16;  
  
let tieneMembresia = false;  
  
let estaAcompañadoPorAdulto = true;  
  
console.log((edad >= 18 && tieneMembresia) || estaAcompañadoPorAdulto);  
  
// true, porque aunque no cumple las primeras dos condiciones, está acompañado por un adulto
```

Los **operadores de comparación y lógicos** son esenciales para evaluar condiciones y combinar reglas en cualquier programa. Aunque JavaScript te permite usar `==` para comparar valores con tipos diferentes, es mejor usar `===` para evitar conversiones automáticas que puedan generar errores. Al dominar estos operadores, podrás construir lógica más robusta y clara en tus programas.

Estructura de Control if

Hasta ahora hemos aprendido cómo comparar valores usando **operadores de comparación** y cómo combinar condiciones con **operadores lógicos**. En esta lectura, vamos a ver cómo usar estas comparaciones para tomar decisiones dentro de nuestros programas. Para esto, usaremos una herramienta muy importante en JavaScript: la **estructura de control if**.

¿Qué es un if?

Un **if** es una estructura que nos permite tomar decisiones en el código. Funciona de una manera muy simple: "Si una condición es verdadera, haz esto. Si no, haz otra cosa."

Sintaxis básica de if:

```
if (condición) {  
    // código que se ejecuta si la condición es verdadera  
}
```

Ejemplo sencillo de if

Imagina que tienes un programa que necesita verificar si una persona tiene la edad suficiente para votar. Si la persona tiene 18 años o más, puede votar. Podemos expresar esto con un **if**:

```
let edad = 20;  
  
if (edad >= 18) {  
    console.log("Puedes votar");  
}
```

En este caso, la condición `edad >= 18` es true, porque **20 es mayor que 18**. Como la condición es verdadera, el programa ejecuta el código dentro del if y muestra "Puedes votar".

¿Y si la condición no es verdadera?: else

A veces necesitamos que el programa haga algo diferente si la condición no es verdadera. Para eso usamos la palabra clave **else**. Esto significa "**si la primera condición no se cumple, ejecuta este otro código**."

Sintaxis de if con else:

```
if (condición) {  
    // código que se ejecuta si la condición es verdadera  
} else {  
    // código que se ejecuta si la condición es falsa  
}
```

Ejemplo con if y else

Volvamos al ejemplo del voto. Ahora, queremos que el programa también diga que no puedes votar si tienes menos de 18 años:

```
let edad = 16;  
  
if (edad >= 18) {  
    console.log("Puedes votar");  
} else {  
    console.log("No puedes votar");  
}
```

En este caso, `edad >= 18` es **false**, porque 16 es menor que 18. Como **la condición no se cumple**, el programa ejecuta el código dentro del bloque `else`, que muestra "No puedes votar".

Múltiples condiciones con else if

A veces queremos evaluar más de una condición antes de tomar una decisión. Para eso, usamos `else if`. Esto nos permite verificar varias condiciones diferentes y ejecutar diferentes bloques de código dependiendo de cuál sea verdadera.

Sintaxis de if con else if y else:

```
if (condición1) {  
    // código que se ejecuta si condición1 es verdadera  
} else if (condición2) {  
    // código que se ejecuta si condición1 es falsa pero condición2 es verdadera  
} else {  
    // código que se ejecuta si ninguna de las condiciones es verdadera  
}
```

```
}
```

Ejemplo con else if

Imagina que quieres verificar si una persona es menor de edad, si es mayor de edad o si es un adulto mayor:

```
let edad = 70;

if (edad < 18) {

    console.log("Eres menor de edad");

} else if (edad >= 18 && edad < 65) {

    console.log("Eres mayor de edad");

} else {

    console.log("Eres un adulto mayor");

}
```

En este caso, `edad < 18` es `false`, porque la persona tiene 70 años. Luego, la segunda condición `edad >= 18 && edad < 65` también es `false`. Finalmente, el programa llega al bloque `else`, donde se ejecuta "Eres un adulto mayor".

Usando operadores lógicos con if

Podemos combinar las condiciones dentro de un `if` usando operadores lógicos como `&&` y `||`. Esto nos permite evaluar varias condiciones a la vez.

Ejemplo:

```
let edad = 20;

let tieneLicencia = true;

if (edad >= 18 && tieneLicencia) {

    console.log("Puedes conducir");

} else {

    console.log("No puedes conducir");

}
```

En este ejemplo, el programa solo imprimirá "Puedes conducir" si ambas condiciones son verdaderas: que la persona tenga más de 18 años y que tenga licencia.

Resumen: ¿Qué hemos aprendido?

La estructura de control **if** es una de las herramientas más útiles en la programación porque nos permite tomar decisiones en nuestros programas. Con **if**, **else if** y **else**, podemos crear reglas y condiciones que el programa debe seguir.

- **if**: Se ejecuta cuando una condición es verdadera.
- **else if**: Se usa para agregar más condiciones.
- **else**: Se ejecuta si ninguna de las condiciones anteriores es verdadera.
- Podemos combinar estas condiciones usando operadores lógicos como **&&** (y) y **||** (o).

Ahora que entendemos cómo usar la estructura de control **if**, podemos empezar a crear programas que reaccionen a diferentes situaciones. La capacidad de tomar decisiones basadas en condiciones nos permitirá escribir código dinámico y adaptable. Practica creando tus propias condiciones y probando diferentes resultados, ya que esta es una de las habilidades más importantes que desarrollarás como programador.